

(7) Networks

- **Graph type:** `nx.Graph` vs `nx.DiGraph` (undirected vs directed)
- **Weights:** edge attribute name (commonly `"weight"`)
- **Connectivity:**
 - undirected: connected components
 - directed: weakly/strongly connected components
- **“Documented outputs”:** which functions print vs return

Create networks with node and edge list

```
# 1.1

# `nodelist.tsv` a tab-separated file containing the nodes of the graph.
Recall that each node represents a different university. The file has
these columns:
# `u` – the node index;      – `score` – the rank of the university (the
higher the better);      – `name` – the name of the university.
# `edgelist.tsv` a tab-separated file containing the edges of the graph.
Recall that each edge represents a hiring decision. The file has columns:
# `u` – the source university, where the person got their Ph.D.;      – `v`
– the target university, where the person was hired;      – `gender` – the
gender of the person.

import networkx as nx
import pandas as pd

# Loads the graph

# Comment: This graph should have been loaded as a multigraph, this is
because there may be multiple edges between
# the same two nodes. Loading this as a DiGraph collapsed these multiple
edges. We choose not to penalize students
# that loaded the graph as a DiGraph, even though results differed
slightly!
G = nx.MultiDiGraph()
edge_list = pd.read_csv("../data/part-1/edgelist.tsv", sep="\t")
node_list = pd.read_csv("../data/part-1/nodelist.tsv", sep="\t")

# Creates node attributes
for _, node in node_list.iterrows():
    node = dict(node)
    G.add_node(node['u'], score=node['score'],
name=node['name'])

# Creates edge attributes
```

```

for _, edge in edge_list.iterrows():
    edge = dict(edge)
    G.add_edge(edge['u'], edge['v'],
gender=edge['gender'])
    # Print number of edges
print("Number of nodes", len(G.nodes))
print("Number of edges", len(G.edges))

```

```

...

```

1. Using the above files, create the directed graph representing the hiring network using `networkx`.

Each node should contain the attributes `score` and `name`, and each edge should contain the attribute `gender`.

Print the total numbers of nodes and edges in the graph.

2. Create a plot that allows you to easily visualize ***(a)*** what fraction of all researchers in the graph are hired by the N universities that hire the most; and ***(b)*** what fraction of all researchers in the graph are trained by the N universities whose students get hired the most. Here, N could be any number between 1 and the number of universities.

```

...

```

```

# 1.2

```

```

import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

```

```

# Get indegree and outdegree distributions

```

```

indegree = []
outdegree = []
for node in G.nodes:
    indegree.append(len(list(G.predecessors(node))))
    outdegree.append(len(list(G.successors(node))))
indegree = np.array(indegree)
outdegree = np.array(outdegree)

```

```

indegree = np.array(sorted(indegree/sum(indegree), reverse=True)).cumsum()
outdegree = np.array(sorted(outdegree/sum(outdegree),
reverse=True)).cumsum()

```

```

# Makes ecdfplot

```

```

plt.plot(indegree, label="% Ph.D. students hired\nby the top %N$
universities")
plt.plot(outdegree, label="% Ph.D. students trained\nby the top %N$
universities")
plt.xlabel("%N$")
plt.legend()
plt.show();

```

make triangles also good filtering at the start

```
# 2.1

# Provided code! Do not change!
import networkx as nx
G = nx.from_pandas_edgelist(pd.read_csv("./wiki-RfA.csv.gz"),
    'SRC', 'TGT', ['VOT', 'RES', 'YEA', 'DAT'], create_using=nx.Graph)

import networkx as nx
edges_2004 = [i for i, v in nx.get_edge_attributes(G, "YEA").items() if v
    == 2004]
G_2004 = G.edge_subgraph(edges_2004)

idx = 0
tmp = []
for i in nx.enumerate_all_cliques(G_2004):
    if len(i) < 3:
        continue
    if len(i) > 3:
        break
    idx += 1
    tmp.append(i)

print("Triangles", len(tmp))
```

Goodies

```
G = nx.Graph() # for a directed graph use nx.DiGraph()
G.add_node(1)
G.add_nodes_from(range(2,9)) # add multiple nodes at once

# add edges G.add_edge(1,2)
edges = [(2,3), (1,3), (4,1), (4,5), (5,6), (5,7), (6,7), (7,8), (6,8)]
G.add_edges_from(edges)
G.nodes()
plt.show()

# plot it out
# for different layouts, please see:
https://networkx.github.io/documentation/stable/reference/drawing.html#module-networkx.drawing.layout
nx.draw_spring(G, with_labels=True, alpha = 0.8)
plt.show()
```

```

# Helper function for plotting the degree distribution of a Graph
def plot_degree_distribution(G):
    degrees = {}
    for node in G.nodes():
        degree = G.degree(node)
        if degree not in degrees:
            degrees[degree] = 0
        degrees[degree] += 1
    sorted_degree = sorted(degrees.items())
    deg = [k for (k,v) in sorted_degree]
    cnt = [v for (k,v) in sorted_degree]
    fig, ax = plt.subplots()
    plt.bar(deg, cnt, width=0.80, color='b')
    plt.title("Degree Distribution")
    plt.ylabel("Frequency")
    plt.xlabel("Degree")
    ax.set_xticks([d+0.05 for d in deg])
    ax.set_xticklabels(deg)


# Helper function for printing various graph properties
def describe_graph(G):
    print(G)
    if nx.is_connected(G):
        print("Avg. Shortest Path Length: %.4f"
              %nx.average_shortest_path_length(G))
        print("Diameter: %.4f" %nx.diameter(G)) # Longest shortest path
    else:
        print("Graph is not connected")
        print("Diameter and Avg shortest path length are not defined!")
    print("Sparsity: %.4f" %nx.density(G)) # #edges/#edges-complete-graph
    # #closed-triplets(3*#triangles)/#all-triplets    print("Global
clustering coefficient aka Transitivity: %.4f" %nx.transitivity(G))

...

Graph with 10 nodes and 20 edges
Avg. Shortest Path Length: 1.6000
Diameter: 3.0000
Sparsity: 0.4444
Global clustering coefficient aka Transitivity: 0.4853
...


# Helper function for visualizing the graph
def visualize_graph(G, with_labels=True, k=None, alpha=1.0,
node_shape='o'):

```

```

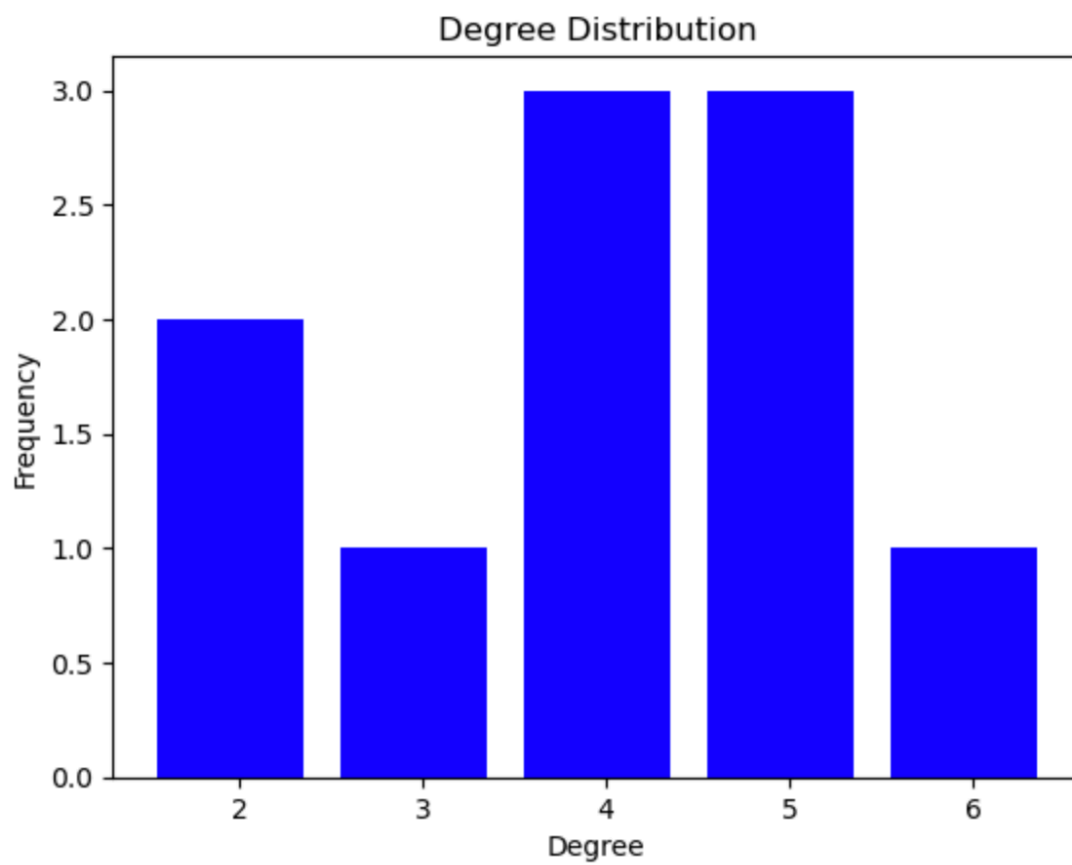
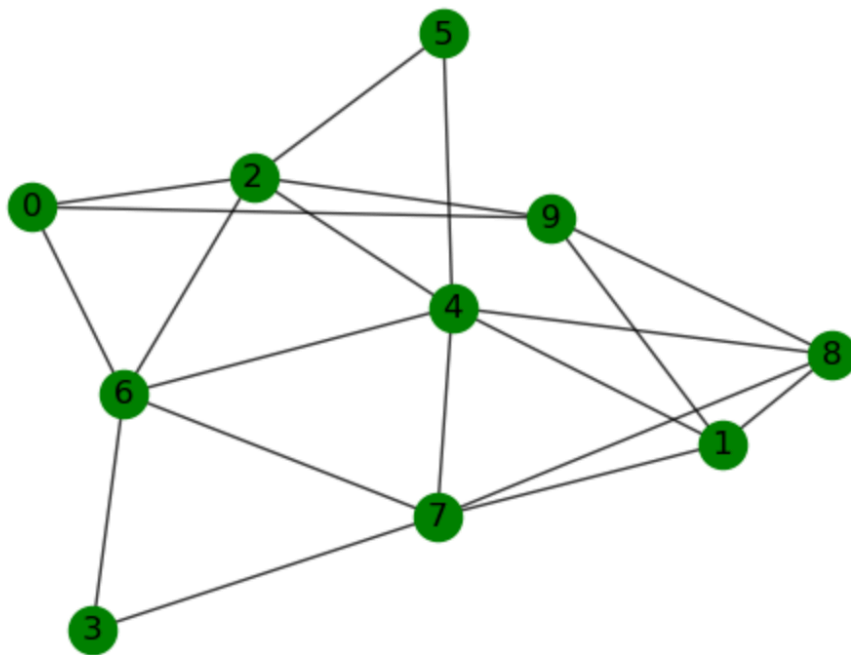
#nx.draw_spring(G, with_labels=with_labels, alpha = alpha)
pos = nx.spring_layout(G, k=k)
if with_labels:
    lab = nx.draw_networkx_labels(G, pos, labels=dict([(n, n) for n in
G.nodes()])))
    ec = nx.draw_networkx_edges(G, pos, alpha=alpha)
    nc = nx.draw_networkx_nodes(G, pos, nodelist=G.nodes(),
node_color='g', node_shape=node_shape)
    plt.axis('off')

n = 10 # 10 nodes
m = 20 # 20 edges

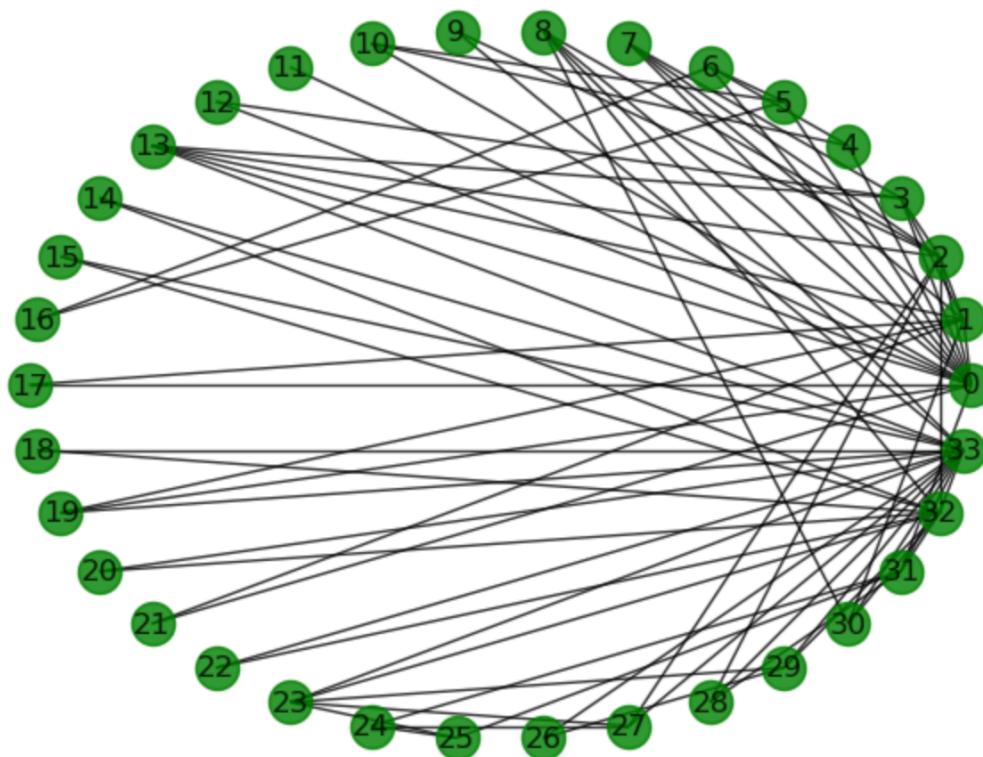
erG = nx.gnm_random_graph(n, m)

describe_graph(erG)
visualize_graph(erG, k=0.05, alpha=0.8)
plot_degree_distribution(erG)
plt.show()

```



```
# Draw the graph with a circular layout instead?
nx.draw_circular(karateG, with_labels=True, node_color='g', alpha = 0.8)
plt.show()
```



Load CSV edge list and create a network from pandas

```
# let's see which quaker knows whom, this will translate into edges in our
graph
edges = pd.read_csv(data_folder + 'quakers_edgelist.csv')
edges.head()

quakerG = nx.from_pandas_edgelist(edges, 'Source', 'Target',
edge_attr=None, create_using= nx.Graph())
describe_graph(quakerG)

# add node attributes by passing dictionary of type name -> attribute
nx.set_node_attributes(quakerG, nodes['Role'].to_dict(), 'Role' )
nx.set_node_attributes(quakerG, nodes['Gender'].to_dict(), 'Gender' )
nx.set_node_attributes(quakerG, nodes['Birthdate'].to_dict(), 'Birthdate'
)
nx.set_node_attributes(quakerG, nodes['Deathdate'].to_dict(), 'Deathdate'
)
nx.set_node_attributes(quakerG, nodes['Quaker'].to_dict(), 'Quaker' )

# You can easily get the attributes of a node
quakerG.nodes['William Penn']

{'Role': 'Quaker leader and founder of Pennsylvania',
 'Gender': 'male',
 'Birthdate': 1644,
 'Deathdate': 1718,
```

```
'Quaker': True}
```

Components

```
# Network Sparsity
# 174 * (2) / ( 119* 118)
print("Network sparsity: %.4f" %nx.density(quakerG))

print(nx.is_connected(quakerG))
comp = list(nx.connected_components(quakerG))
print('The graph contains', len(comp), 'connected components')
# False
# The graph contains 12 connected components

largest_comp = max(comp, key=len)
percentage_lcc = len(largest_comp)/quakerG.number_of_nodes() * 100
print('The largest component has', len(largest_comp), 'nodes', 'accounting
for %.2f'% percentage_lcc, '% of the nodes')
# The largest component has 96 nodes accounting for 80.67 % of the nodes
```

Diameter and Shortest Paths

Suppose I want to find the shortest path between two quakers, given that they are in the same connected component

```
fell_whitehead_path = nx.shortest_path(quakerG, source="Margaret Fell",
target="George Whitehead")
print("Shortest path between Fell and Whitehead:", fell_whitehead_path)

#Shortest path between Fell and Whitehead: ['Margaret Fell', 'George Fox',
'George Whitehead']

# take the largest component and analyse its diameter = longest shortest-
path
lcc_quakerG = quakerG.subgraph(largest_comp)
print("The diameter of the largest connected component is",
nx.diameter(lcc_quakerG))
print("The avg shortest path length of the largest connected component
is", nx.average_shortest_path_length(lcc_quakerG))

# The diameter of the largest connected component is 8
# The avg shortest path length of the largest connected component is
3.3789473684210525

# Employ a **global** measure called **transitivity** (aka global
```



```

clustering coefficient), or the ratio of all existing triangles (closed
triples) over all # possible triangles (open and closed triplets).
print('%.4f' %nx.transitivity(quakerG))
#0.1694

# Employ a **local** measure called **clustering coefficient**, which
quantifies for a node how close its neighbours are to being a clique
(complete graph). Measured as the ratio of, the number of edges to the
number of all possible edges, among the neighbors of a node.
# Similar measure but for individual nodes called clustering coefficient
print(nx.clustering(quakerG, ['Alexander Parker', 'John Crook']))
# {'Alexander Parker': 0.06666666666666667, 'John Crook':
0.8333333333333334}

```

Degree: the more people you know, the more important you are!

```

degrees = dict(quakerG.degree(quakerG.nodes()))
sorted_degree = sorted(degrees.items(), key=itemgetter(1), reverse=True)

# And the top 5 most popular quakers are.. for quaker, degree in
sorted_degree[:5]:
    print(quaker, 'who is', quakerG.nodes[quaker]['Role'], 'knows',
degree, 'people')

degree_seq = [d[1] for d in sorted_degree]
degreeCount = collections.Counter(degree_seq)
degreeCount = pd.DataFrame.from_dict( degreeCount,
orient='index').reset_index()
fig = plt.figure()
ax = plt.gca()
ax.plot(degreeCount['index'], degreeCount[0], 'o', c='blue', markersize=
4)
plt.ylabel('Frequency')
plt.xlabel('Degree')
plt.title('Degree distribution for the Quaker network')
plt.show()

```

Other good things that you can find in networkz exercise

1 What about the Katz Centrality (the generalization over degree centrality)?

2 Betweenness centrality: the more shortest paths pass through a node, the more important it is!

3 The quaker communities

Community detection is a common class of methods applied to graphs.

Two important algorithms:

- Girvan Newman
- Louvain

4 Homophily in quakers

How likely is it that two quakers who have the same attribute are linked?

Try to measure the similarity of connections in the graph with respect to a given attribute.

Intuition: Like correlation, but translated to graphs.

Super-CheatCodes - Very General

Add a “safe” graph summary that works for undirected/directed/disconnected

Keep your `describe_graph()` as-is, but add a **new** function that’s robust and returns a dict (easier to reuse).

```
import networkx as nx
import pandas as pd

def graph_summary(G, weight=None):
    """
    Robust summary for undirected/directed graphs.
    – For disconnected graphs: compute path stats on largest component.
    – For DiGraph: uses weak connectivity for components; path stats on
    largest weakly-CC.
    Returns a dict (easy to convert to DataFrame).
    """
    summary = {
        "n_nodes": G.number_of_nodes(),
        "n_edges": G.number_of_edges(),
        "density": nx.density(G),
        "is_directed": G.is_directed(),
    }

    # Components
    if G.is_directed():
        comps = list(nx.weakly_connected_components(G)) if
summary["n_nodes"] else []
    summary["n_components_weak"] = len(comps)
    summary["largest_comp_size"] = max((len(c) for c in comps),
```

```

default=0)
    H = G.subgraph(max(comps, key=len)) if comps else G
    else:
        comps = list(nx.connected_components(G)) if summary["n_nodes"]
else []
    summary["n_components"] = len(comps)
    summary["largest_comp_size"] = max((len(c) for c in comps),
default=0)
    H = G.subgraph(max(comps, key=len)) if comps else G

    # Clustering / transitivity (well-defined for undirected; for directed
    it's trickier)
    try:
        summary["transitivity"] = nx.transitivity(G.to_undirected() if
G.is_directed() else G)
    except Exception:
        summary["transitivity"] = None

    # Path stats (only meaningful if the selected component has >1 node)
    if H.number_of_nodes() > 1:
        UG = H.to_undirected() if H.is_directed() else H
        if nx.is_connected(UG):
            summary["avg_shortest_path_len_LCC"] =
nx.average_shortest_path_length(UG, weight=weight)
            summary["diameter_LCC"] = nx.diameter(UG)
        else:
            summary["avg_shortest_path_len_LCC"] = None
            summary["diameter_LCC"] = None
    else:
        summary["avg_shortest_path_len_LCC"] = None
        summary["diameter_LCC"] = None

    return summary

# Example usage:
# pd.DataFrame([graph_summary(quakerG)])

```

Add a “centrality table” helper (one function, many metrics)

You’ll often want: degree / betweenness / closeness / pagerank in one place.

```

def centrality_table(G, weight=None, k_betweenness=None,
pagerank_alpha=0.85):
    """
    Returns a DataFrame with common centralities.
    - k_betweenness: if set (int), uses approximation for betweenness
    (faster on large graphs).
    """
    df = pd.DataFrame(index=G.nodes())

```

```

df["degree"] = pd.Series(dict(G.degree(weight=weight)))

# Closeness: for directed graphs NetworkX defines variants; converting
to undirected is common
UG = G.to_undirected() if G.is_directed() else G
df["closeness"] = pd.Series(nx.closeness centrality(UG))

if k_betweenness is None:
    df["betweenness"] = pd.Series(nx.betweenness centrality(UG,
weight=weight, normalized=True))
else:
    df["betweenness"] = pd.Series(nx.betweenness centrality(UG,
k=k_betweenness, weight=weight, normalized=True, seed=0))

# PageRank: meaningful especially for directed graphs
if G.is_directed():
    df["pagerank"] = pd.Series(nx.pagerank(G, alpha=pagerank_alpha,
weight=weight))
else:
    df["pagerank"] = pd.Series(nx.pagerank(G.to_directed(),
alpha=pagerank_alpha, weight=weight))

return df.sort_values("pagerank", ascending=False)

```

Add a degree distribution helper with log-log / CCDF option

```

import numpy as np
import matplotlib.pyplot as plt
from collections import Counter

def plot_degree_ccdf(G):
    """
    Plots CCDF of degree distribution on log-log axes.
    Useful for heavy-tailed networks.
    """
    degrees = [d for _, d in G.degree()]
    cnt = Counter(degrees)
    xs = np.array(sorted(cnt.keys()))
    pmf = np.array([cnt[x] for x in xs], dtype=float) / len(degrees)
    ccdf = 1.0 - np.cumsum(pmf) + pmf # P(D >= x)

    plt.figure(figsize=(6,4))
    plt.plot(xs, ccdf, marker="o", linestyle="None")
    plt.xscale("log")
    plt.yscale("log")
    plt.xlabel("Degree (log)")
    plt.ylabel("CCDF P(D ≥ k) (log)")
    plt.title("Degree CCDF")

```

```
plt.show()
```

Create MultiDiGraph from df data exam 2023

```
id_to_speaker = dict(zip(df["id"], df["speaker"]))

G = nx.MultiDiGraph()

# keep only rows that are replies
df_replies = df[df["reply-to"].notna()]

# add all speakers as nodes
G.add_nodes_from(df["speaker"].unique())

for _, row in df_replies.iterrows():
    source = row["speaker"]
    target = id_to_speaker.get(row["reply-to"])

    # safety check (in case reply-to points to a removed row)
    if target is None:
        continue

    G.add_edge(
        source,
        target,
        season=row["season"],
        episode=row["episode"]
    )

print(G)
```

Access + read and get some stuff, out-degree and prints

```
import networkx as nx

G = nx.read_graphml("data/exam2.graphml")

out_degree = dict(G.out_degree())

pagerank = nx.pagerank(G)

MCS = [
    "Joey Tribbiani",
    "Monica Geller",
    "Chandler Bing",
    "Phoebe Buffay",
    "Ross Geller",
```

```

    "Rachel Green"
]

print("Out-degree:")
for c in MCS:
    print(f"{c}: {out_degree.get(c, 0)}")

print("\nPageRank:")
for c in MCS:
    print(f"{c}: {pagerank.get(c, 0):.4f}")

```

general ops, iter nodes/edge etc...

```

"""
=====
=====
COMMON NETWORKX PATTERNS – ITERATION, ATTRIBUTES, AND BASIC MANIPULATION
=====
=====

This section collects the most common NetworkX idioms you will repeatedly
use
in exams and projects. The focus is on:

• Iterating over nodes and edges
• Accessing node and edge attributes
• Inspecting what attributes exist
• Safely handling different graph types (Graph / DiGraph / MultiDiGraph)
• Basic graph manipulation patterns

All examples assume familiarity with nx.Graph, nx.DiGraph,
nx.MultiDiGraph.

=====
=====
"""

import networkx as nx

# -----
# -----
# 1) BASIC GRAPH CREATION (UNDIRECTED / DIRECTED / MULTI)
# -----
# -----

G_undirected = nx.Graph()
G_directed   = nx.DiGraph()
G_multi      = nx.MultiDiGraph()

```

```

# Add nodes with attributes
G_directed.add_node("A", score=10, name="University A")
G_directed.add_node("B", score=20, name="University B")

# Add edges with attributes
G_directed.add_edge("A", "B", gender="F", year=2020)
G_directed.add_edge("B", "A", gender="M", year=2021)

print(G_directed)
# DiGraph with 2 nodes and 2 edges

# -----
# -----
# 2) ITERATING OVER NODES
# -----

"""
G.nodes can be viewed in three useful ways:

1) G.nodes()           → node IDs only
2) G.nodes(data=True)  → (node, attr_dict)
3) G.nodes[node_id]    → attribute dict for a single node
"""

# 2.1 Node IDs only
for node in G_directed.nodes():
    print("Node:", node)

# 2.2 Nodes with attributes
for node, attrs in G_directed.nodes(data=True):
    print(f"Node {node} has attributes:", attrs)

# 2.3 Access attributes of a specific node
print("Attributes of node A:", G_directed.nodes["A"])
print("Score of A:", G_directed.nodes["A"]["score"])

# Safe access (no KeyError)
print("Ranking of A:", G_directed.nodes["A"].get("ranking", None))

# -----
# -----
# 3) ITERATING OVER EDGES
# -----

"""

```

Edge iteration depends on graph type.

Graph / DiGraph:

(u, v) or (u, v, data)

MultiGraph / MultiDiGraph:

(u, v, key, data)

"""

3.1 Edges without attributes

```
for u, v in G_directed.edges():  
    print(f"Edge {u} -> {v}")
```

3.2 Edges with attributes

```
for u, v, attrs in G_directed.edges(data=True):  
    print(f"Edge {u} -> {v} has attributes:", attrs)
```

3.3 MultiDiGraph edge iteration

```
G_multi.add_edge("A", "B", gender="F")
```

```
G_multi.add_edge("A", "B", gender="M")
```

```
for u, v, key, attrs in G_multi.edges(keys=True, data=True):  
    print(f"Edge {u} -> {v} (key={key}) attrs:", attrs)
```

#

4) GETTING ALL ATTRIBUTE NAMES PRESENT IN THE GRAPH

#

"""

Often you don't know which attributes exist.
You can collect them by iterating once.

"""

Node attribute names

```
node_attr_names = set()
```

```
for _, attrs in G_directed.nodes(data=True):  
    node_attr_names.update(attrs.keys())
```

```
print("Node attributes present:", node_attr_names)
```

Edge attribute names

```
edge_attr_names = set()
```

```
for _, _, attrs in G_directed.edges(data=True):  
    edge_attr_names.update(attrs.keys())
```

```
print("Edge attributes present:", edge_attr_names)
```



```

# -----
# 5) ACCESSING ATTRIBUTES VIA DICTIONARY VIEWS (FAST)
# -----

"""
NetworkX provides dictionary-like views for attributes.
These are often faster and cleaner.
"""

# All node attributes of one type
scores = nx.get_node_attributes(G_directed, "score")
print("Node scores:", scores)

# All edge attributes of one type
genders = nx.get_edge_attributes(G_directed, "gender")
print("Edge genders:", genders)

# -----
# 6) DEGREE, IN-DEGREE, OUT-DEGREE
# -----

"""
Degree depends on graph type:
- Graph: degree
- DiGraph: in_degree, out_degree
"""

print("Degrees:", dict(G_directed.degree()))
print("In-degrees:", dict(G_directed.in_degree()))
print("Out-degrees:", dict(G_directed.out_degree()))

# -----
# 7) FILTERING NODES OR EDGES BY ATTRIBUTE
# -----

# Nodes with score >= 15
high_score_nodes = [
    n for n, attrs in G_directed.nodes(data=True)
    if attrs.get("score", 0) >= 15
]

```

```

print("High-score nodes:", high_score_nodes)

# Edges with female hires
female_edges = [
    (u, v) for u, v, attrs in G_directed.edges(data=True)
    if attrs.get("gender") == "F"
]

print("Female edges:", female_edges)

# -----
# 8) SUBGRAPHS (FILTERED GRAPHS)
# -----

# Induced subgraph on nodes
H_nodes = G_directed.subgraph(high_score_nodes)
print("Subgraph (nodes):", H_nodes)

# Edge-induced subgraph
H_edges = G_directed.edge_subgraph(female_edges)
print("Subgraph (edges):", H_edges)

# -----
# 9) MODIFYING ATTRIBUTES IN PLACE
# -----

# Update node attribute
for node in G_directed.nodes():
    G_directed.nodes[node]["active"] = True

# Update edge attribute
for u, v, attrs in G_directed.edges(data=True):
    attrs["weight"] = 1.0

print("Updated node attributes:", dict(G_directed.nodes(data=True)))
print("Updated edge attributes:", list(G_directed.edges(data=True)))

# -----
# 10) SAFE GRAPH COPYING AND TYPE CONVERSION
# -----

```

```

# Shallow copy
G_copy = G_directed.copy()

# Convert to undirected (drops direction)
G_undirected_from_directed = G_directed.to_undirected()

# Convert to directed
G_directed_from_undirected = G_undirected.to_directed()

# -----
# -----
# 11) QUICK GRAPH INSPECTION (NO COMPUTATION)
# -----
# -----

print("Is directed?", G_directed.is_directed())
print("Nodes:", G_directed.number_of_nodes())
print("Edges:", G_directed.number_of_edges())

# -----
# -----
# 12) MENTAL MODEL (IMPORTANT)
# -----
# -----

"""
• Nodes and edges are stored as dictionaries
• Iteration gives you IDs + attribute dicts
• Graph views are lightweight and fast
• MultiGraphs add a 'key' dimension to edges
• Always check graph type before using degree / components / paths

If you remember:
    for n, attrs in G.nodes(data=True)
    for u, v, attrs in G.edges(data=True)

you can access almost everything in NetworkX.
"""

```

